

Celebal Technologies Internship

Week 2 SQL Assignment

Name: Rishi Jain

Ques 1 output

```
3  -- Section A - SQL Basics (SELECT, Constraints, Primary Keys)
4
5  -- Q1
6  • select * from customers;
```

Result Grid

customer_id	first_name	last_name	email	city	state	join
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	202
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	202
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	202
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	202
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	202
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	202
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	202

customers 1 x

Apply Revert

Que 2

```
3  -- Q2
4  • select first_name,last_name,city from customers;
```

Result Grid

first_name	last_name	city
Aarav	Sharma	Mumbai
Priya	Patel	Ahmedabad
Rohan	Gupta	Delhi
Sneha	Reddy	Hyderabad
Vikram	Singh	Jaipur
Ananya	Iyer	Chennai
Karan	Mehta	Pune

customers 2 x

Read Only

Que 3

```

10
11 -- Q3
12 • select distinct category from products;

```

Result Grid

category
Clothing
Electronics
Home

products 3 x Read Only

Que 7

```

62 Execute the selected portion of the script or everything, if there is no selection
63 -- Q7
64 • select * from orders
65 where status = 'Delivered';
66
67 -- Q8

```

Result Grid

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.00
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1008	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1598.00
NULL	NULL	NULL	NULL	NULL

Que 8

```

67 -- Q8
68 • select * from products
69 where category = 'electronics' and unit_price > 2000;
70
71 -- Q9
72 • select * from customers

```

Result Grid

product_id	product_name	category	brand	unit_price	stock_qty
203	Smart Watch	Electronics	Noise	2999.00	150
205	Bluetooth Speaker	Electronics	JBL	3499.00	200
NULL	NULL	NULL	NULL	NULL	NULL

Form Editor

Que 9

```

70
71 -- Q9
72 • select * from customers
73 where year(join_date) = 2024 and state = 'maharashtra';
74
75 -- Q10
76

```

Limit to 1000 rows

Result Grid

customer_id	first_name	last_name	email	city	state	join_date
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-
107	Karan	Mehra	karan.m@email.com	Pune	Maharashtra	2024-07-
NULL	NULL	NULL	NULL	NULL	NULL	NULL

customers 6 x Apply Revert Cont

Que 10

Limit to 1000 rows

```
-- Q10
select * from orders
where order_date between '2024-08-10' and '2024-08-25'
and status <> 'cancelled';
```

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00
NULL	NULL	NULL	NULL	NULL

orders 7 x

Apply Revert Context Help

Que 13

Limit to 1000 rows

```
-- Execute the selected portion of the script or everything, if there is no selection
select count(*) as total_orders from orders;

-- Q14
select sum(total_amount) from orders where status = 'delivered';
```

total_orders
12

Result 8 x

Read Only

Que 14

Execute the selected portion of the script or everything, if there is no selection

```
-- Q15
select sum(total_amount) from orders where status = 'delivered';

-- Q15
select category, avg(unit_price) from products group by category;
```

sum(total_amount)
17191.00

Result Grid

Form Editor

Que 15

```
119 -- Q14
120
121 • select sum(total_amount) from orders where status = 'delivered';
122
123 -- Q15
124 • select category,avg(unit_price) from products group by category;
```

Result Grid

	category	avg(unit_price)
▶	Clothing	2695
	Electronics	2224.000000
	Home	949.000000

Refresh data re-executing the original query

Result 10 x Read Only

que 16

```
124 • select category,avg(unit_price) from products group by category;
```

```
125
126 -- Q16
127 • select status, count(*) as order_count,sum(total_amount) as total_revenue
128 from orders
129 group by status
130 order by total_revenue desc;
```

Result Grid

	status	order_count	total_revenue
▶	Delivered	6	17191.00
	Shipped	2	13596.00
	Pending	3	4495.00
	Cancelled	1	2999.00

Result 11 x Read Only

que 17

```
129 :status
130 :total_revenue desc;
```

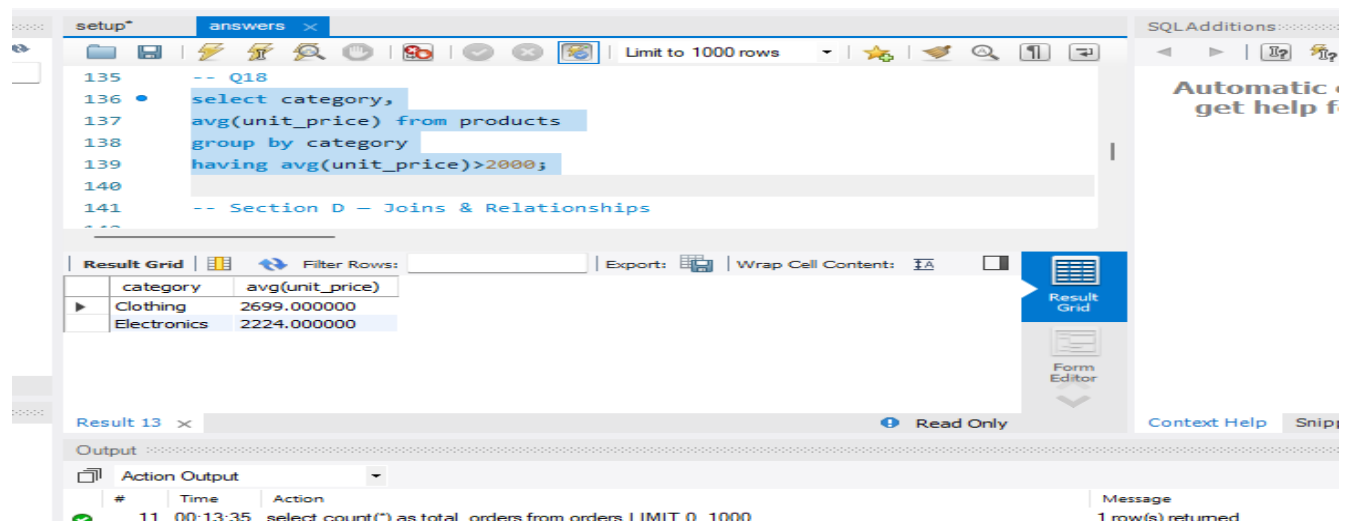
```
131
132
133 • category,max(unit_price),min(unit_price) from products group by category;
```

Result Grid

	category	max(unit_price)	min(unit_price)
▶	Clothing	4599.00	799.00
	Electronics	3499.00	899.00
	Home	1299.00	599.00

Result 12 x Read Only

Que 18



SQL Developer interface showing the execution of a query for Question 18. The query is:

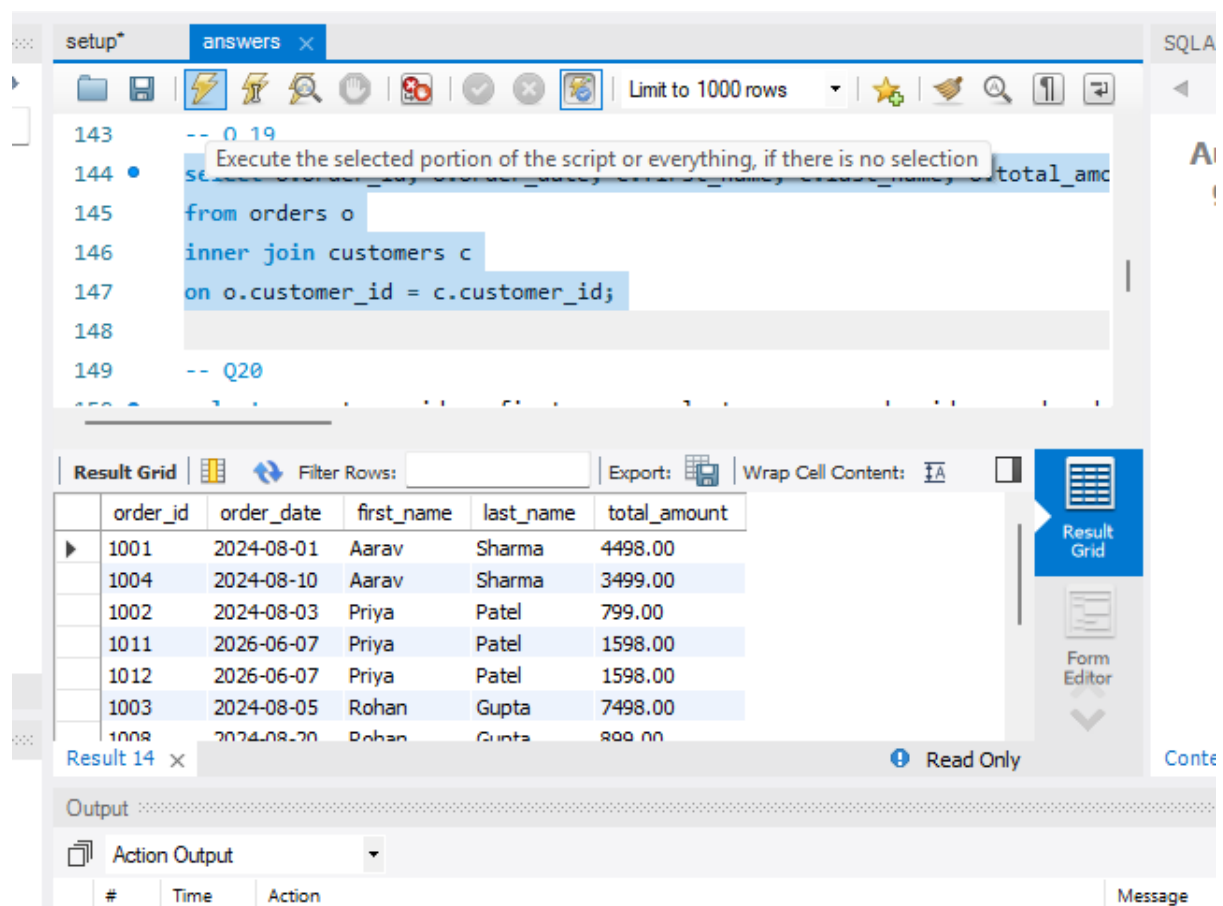
```
-- Q18
select category,
avg(unit_price) from products
group by category
having avg(unit_price)>2000;
```

The result grid displays the following data:

category	avg(unit_price)
Clothing	2699.000000
Electronics	2224.000000

The bottom panel shows the output of the query, indicating that 1 row(s) returned.

Que 19



SQL Developer interface showing the execution of a query for Question 19. The query is:

```
-- Q19
select order_id, order_date, first_name, last_name, total_amount
from orders o
inner join customers c
on o.customer_id = c.customer_id;
```

The result grid displays the following data:

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1011	2026-06-07	Priya	Patel	1598.00
1012	2026-06-07	Priya	Patel	1598.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	800.00

The bottom panel shows the output of the query, indicating that 1 row(s) returned.

Que 20

The screenshot shows the SQLAddict interface with a SQL query editor and a result grid. The query is for Question 20, which asks for a list of customers and their orders.

```
-- Q20
select c.customer_id, c.first_name, c.last_name, o.order_id, o.order_date
from customers c
left join orders o
on c.customer_id = o.customer_id;
```

The result grid displays the following data:

customer_id	first_name	last_name	order_id	order_date
101	Aarav	Sharma	1001	2024-08-01
101	Aarav	Sharma	1004	2024-08-10
102	Priya	Patel	1002	2024-08-03
102	Priya	Patel	1011	2026-06-07
102	Priya	Patel	1012	2026-06-07
103	Rohan	Gupta	1003	2024-08-05
103	Rohan	Gupta	1008	2024-08-20

Que 21

The screenshot shows the SQLAddict interface with a SQL query editor and a result grid. The query is for Question 21, which asks for a list of orders and their items.

```
-- Q21
select o.order_id, p.product_name, oi.quantity, oi.unit_price, oi.discount_pct
from orders o
join order_items oi
on o.order_id = oi.order_id
join products p
on oi.product_id = p.product_id;
```

The result grid displays the following data:

order_id	product_name	quantity	unit_price	discount_pct
1001	Wireless Earbuds	2	1499.00	0.00
1006	Wireless Earbuds	1	1499.00	10.00
1002	Cotton T-Shirt	1	799.00	0.00
1003	Smart Watch	1	2999.00	0.00
1005	Smart Watch	1	2999.00	0.00
1003	Running Shoes	1	4599.00	5.00
1006	Running Shoes	1	4599.00	5.00

Que 24

The screenshot shows the SQLAddict interface with a SQL query editor and a result grid. The query is for Question 24, which asks for a list of products and their price tiers.

```
-- Q24
select product_name,
unit_price,
case
when unit_price < 1000 then 'Budget'
when unit_price between 1000 and 3000 then 'Mid-Range'
when unit_price > 3000 then 'Premium'
end as price_tier
from products;
```

The result grid displays the following data:

product_name	unit_price	price_tier
Wireless Earbuds	1499.00	Mid-Range
Cotton T-Shirt	799.00	Budget
Smart Watch	2999.00	Mid-Range
Running Shoes	4599.00	Premium
Bluetooth Speaker	3499.00	Premium
Bedsheet Set	1299.00	Mid-Range
Anten Stand	899.00	Budget

Que 25

SQL Developer interface showing a query execution for Que 25. The query is:

```
-- Q25
select
count(case when status = 'Delivered' then 1 end) as delivered_orders,
count(case when status <> 'Delivered' then 1 end) as not_delivered_orders
from orders;
```

The Result Grid shows the following data:

delivered_orders	not_delivered_orders
6	6

Additional UI elements include a toolbar with icons for saving, running, and other database actions, and a sidebar with 'Result Grid' and 'Form Editor' buttons.

Que 27

SQL Developer interface showing a transaction execution for Que 27. The transaction is:

```
-- Q27
start transaction;
insert into orders
values (1020, 102, curdate(), 'Pending', 1598.00);
insert into order_items
values
(5020, 1020, 206, 1, 1299.00, 0),
(5021, 1020, 208, 1, 599.00, 0);
update products
set stock_qty = stock_qty - 1
where product_id = 206;
update products
set stock_qty = stock_qty - 1
where product_id = 208;
```

The Output window shows the following actions and their results:

#	Time	Action	Message	Duration / Fetch
48	00:19:23	start transaction	0 row(s) affected	0.000 sec
49	00:19:23	insert into orders values (1020, 102, curdate(), 'Pending', 1598.00)	1 row(s) affected	0.063 sec
50	00:19:23	insert into order_items values (5020, 1020, 206, 1, 1299.00, 0), (5021, 1020, 208, 1, 599.00, 0)	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.000 sec
51	00:19:23	update products set stock_qty = stock_qty - 1 where product_id = 206	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
52	00:19:23	update products set stock_qty = stock_qty - 1 where product_id = 208	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
53	00:19:23	commit	0 row(s) affected	0.000 sec

The SQL Additions pane on the right displays a message: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."